

A Plan Reuse Mechanism for LLM-Driven Agent

Guopeng Li

University of Science and Technology
of China
Hefei, China

Ruiqi Wu

University of Science and Technology
of China
Hefei, China

Haisheng Tan*

University of Science and Technology
of China
Hefei, China

Abstract

Integrating large language models (LLMs) into personal assistants, like Xiao Ai and Blue Heart V, effectively enhances their ability to interact with humans, solve complex tasks, and manage IoT devices. Such assistants are also termed LLM-driven agents. Upon receiving user requests, the LLM-driven agent generates plans using an LLM, executes these plans through various tools, and then returns the response to the user. During this process, the latency for generating a plan with an LLM can reach tens of seconds, significantly degrading user experience. Real-world dataset analysis shows that about 30% of the requests received by LLM-driven agents are identical or similar, which allows the reuse of previously generated plans to reduce latency. However, it is difficult to accurately define the similarity between the request texts received by the LLM-driven agent through directly evaluating the original request texts. Moreover, the diverse expressions of natural language and the unstructured format of plan texts make implementing plan reuse challenging. To address these issues, we present and implement a plan reuse mechanism for LLM-driven agents called AgentReuse. AgentReuse leverages the similarities and differences among requests' semantics and uses intent classification to evaluate the similarities between requests and enable the reuse of plans. Experimental results based on a real-world dataset demonstrate that AgentReuse achieves a 93% effective plan reuse rate, an F1 score of 0.9718, and an accuracy of 0.9459 in evaluating request similarities, reducing latency by 93.12% compared with baselines without using the reuse mechanism.

Keywords

Large language models (LLMs), Agent, Semantic Cache, Similarity evaluation

1 Introduction

In recent years, with the growth in the number of IoT devices and the rise of technologies such as artificial intelligence (AI) and edge computing, the Artificial Intelligence of Things (AIoT) has received widespread attention as a future trend in IoT development [48]. As typical AIoT applications, personal assistants such as Xiao Ai [55] and Huawei Xiaoyi [18] have improved the utility and integration of IoT devices in fields like smart homes and smart transportation by simplifying the user control process, thereby enhancing the interactivity and automation levels of AIoT systems.

Recently, Large Language Models (LLMs) such as ChatGPT, DeepSeek, and Qwen have demonstrated exceptional capabilities

in understanding and generating human language, bringing new opportunities for the development of personal assistants [10]. Manufacturers like Google, Xiaomi, and VIVO have launched personal assistants based on large models, aiming to leverage LLMs to improve abilities in interacting with humans, completing complex tasks, and efficiently managing and controlling applications and various IoT devices [24]. Such assistants are termed *personal LLM-driven agents* [51], or simply *agents* in this paper.

An LLM-driven agent uses an LLM as its core controller, supplemented by capabilities such as planning, memory, and tool use [49]. By coordinating these different capabilities, it completes tasks submitted by users. When an agent receives a task request from a user, it first utilizes the LLM's planning capability to generate a plan, decomposing a complex task into a series of simple sub-tasks. Then, by invoking tools such as APIs, code interpreters, and search engines, and combining them with the memory possessed by the LLM (such as user personal preferences), it executes the sub-tasks to complete the entire mission.

For example, when a user submits the request "Book a ticket from Hefei to Beijing for the day after tomorrow," the plan generated by the agent using the LLM might be:

- (1) Query the date for the day after tomorrow;
- (2) Query the weather;
- (3) Query flight schedules and prices from Hefei to Beijing;
- (4) Query train schedules and prices from Hefei to Beijing;
- (5) Compare flight and train prices;
- (6) Query the user's personal schedule;
- (7) Combine the above information and user preferences to recommend a travel plan.

After deriving the plan, the agent completes sub-tasks 1, 2, 3, 4, and 6 by calling applications on the mobile phone or open APIs; completes sub-task 5 via a code interpreter; and finally, for sub-task 7, combines weather, price information, personal schedule, and personal preferences stored in memory to obtain the travel plan.

In this example, we can see that agents integrated with LLM capabilities can better complete tasks by combining user personal needs [40, 54]. However, the computation and latency costs of using LLMs cannot be ignored. Using AutoGen [53] as the agent framework and OpenAI's GPT-4 API as the LLM, for the input task "Book a ticket from Hefei to Beijing for the day after tomorrow," the latency for returning the plan is approximately 25 seconds, consuming 2015 tokens, with a cost of \$0.073 (approx. 0.53 RMB). Within the 25-second latency, the plan generation latency is about 24 seconds, with the rest being network latency. This is because LLMs generate plans in an auto-regressive manner, and plan texts are generally long. A plan generation latency of 25 seconds, plus the subsequent execution latency, significantly affects user experience. Analysis of the personal assistant dataset in [13] and [40] indicates that about

*Corresponding author.

This paper is an English version of *A Plan Reuse Mechanism for LLM-Driven Agent* (<https://crad.ict.ac.cn/en/article/doi/10.7544/issn1000-1239.202440380>), published in 2024 in the Journal of Computer Research and Development.

30% of task requests in LLM-driven agents are semantically identical or similar. Therefore, caching the plans generated by the agent using the LLM and reusing previous plans when identical or similar requests arrive subsequently is an effective method to reduce the response latency of LLM-driven agents [60].

Zhou et al. [4] studied caching and reuse methods for the responses of the LLM itself. By vectorizing request texts (embedding) and comparing the semantic similarity between different request texts, it judges whether a subsequent request can reuse the response of a previous request. For example, "Please explain what AIoT is" and "What is AIoT" are semantically similar; one can directly reuse the LLM's response to the former without generating a new introduction, thereby reducing latency.

However, in the process of an LLM-driven agent handling user requests, consider the following two requests:

- Request 1: "Book a ticket from Hefei to Beijing for the day after tomorrow"
- Request 2: "Book a ticket from Changsha to Shanghai for tomorrow"

If GPTCache[4] is directly adopted, and Request 2 is judged similar to Request 1, directly returning Request 1's response would obviously provide wrong ticket information to the user. If Request 2 is judged dissimilar to Request 1, it results in a plan generation latency of about 25 seconds. In fact, upon observation, it is not difficult to find that for Request 1 and Request 2, although the time, origin, and destination differ, the *task type* is the same. Therefore, the plan steps produced by the agent should be the same, with only the specific key parameters ("Time, Origin, Destination") being different. That is, Request 2 should be judged as similar to Request 1, but instead of returning Request 1's response, Request 2 should reuse the *plan* generated by the LLM for Request 1.

This paper conducts research on a plan reuse mechanism for LLM-driven agents. Existing research mainly focuses on the caching and reuse mechanisms for the LLM's direct responses. Our research faces three main challenges:

Challenge 1: How to define if requests are similar. As mentioned, in response caching methods, semantic similarity is defined by vectorizing original texts. However, in LLM-driven agents, Request 1 and Request 2 have different key parameters. Tests show that calculating similarity on raw text (with a threshold of 0.75) judges them as dissimilar, whereas Request 2 can factually reuse the plan for Request 1. Thus, a new method is needed to define similarity between request texts in LLM-driven agents.

Challenge 2: How to identify key parameters. After defining similarity, if two requests are similar, the later request can reuse the plan of the former. During reuse, key parameters must be replaced. In Request 1 and 2, key parameters are time (day after tomorrow vs. tomorrow), origin (Hefei vs. Changsha), and destination (Beijing vs. Shanghai). Intuitively, string matching could replace parameters, but natural language contains synonyms and varying word orders (e.g., Request 2 could be "Book me a ticket, Changsha to Shanghai, tomorrow"), which string matching cannot easily handle. Therefore, a method capable of semantic-aware parameter identification is required.

Challenge 3: How to correctly and effectively reuse plans. Currently, plans output by agents are usually unstructured text.

Even if key parameters are identified, it is difficult to determine which sub-task each parameter should apply to, making effective plan reuse hard. Therefore, a method to convert unstructured text into structured execution plans is needed.

To address the above challenges, this paper proposes **AgentReuse**, a plan reuse mechanism for LLM-driven agents, and implements it with millisecond-level extra latency. The main contributions of this paper are in three aspects:

- (1) Addressing the long response time of LLM-driven agents, we formally describe and analyze the workflow of LLM-driven agents and propose a research approach for reusing plans generated by such agents.
- (2) We propose and implement AgentReuse with low overhead. AgentReuse identifies key parameters by leveraging the semantic differences between request texts. After extracting key parameters, it uses an intent classification-based method to define similarity between user requests using semantic similarity, enabling plan reuse.
- (3) To verify the effectiveness of AgentReuse, we conducted experiments using a real-world dataset. The results show that compared to existing methods, AgentReuse achieves an effective plan reuse rate of 93%. The F1 score for similarity definition increased by 6.8 percentage points, accuracy increased by 13.06 percentage points, and latency was reduced by 93.12%.

2 Background and Related Work

Focusing on the research theme of this paper, this section briefly introduces the background knowledge and recent research progress in two aspects: LLM-driven agents and caching & reuse methods.

2.1 LLM-driven Agents

An agent is an autonomous algorithmic system capable of environmental perception, decision-making, and action execution, and has always been a key focus of research in both academia and industry. The original intention of developing agents was to simulate the intelligent behavior of humans or other organisms, aiming to solve problems or execute tasks automatically. However, traditional agents face major challenges as they usually rely on heuristic rules or are limited to specific environmental constraints, which largely restricts their adaptability and scalability in open and dynamic scenarios [26].

Since Large Language Models (LLMs) have demonstrated excellent capabilities in solving complex tasks, an increasing number of research works have begun to explore using LLMs as the control center for agents to improve their performance in open domains and dynamic environments [35], such as the series of LLM-driven agents represented by AutoGPT [35], AutoGen [53], HuggingGPT [44], and MetaGPT [16].

For LLM-driven agents, researchers propose that $Agent = LLM + Memory + Planning + Tool Use$. That is, using the LLM as the core controller, supplemented by capabilities such as planning, memory, and tool usage, the agent completes tasks submitted by users by coordinating these different capabilities [49]. Hong et al. [16] designed MetaGPT, which assigns divisions of labor to multiple agents to jointly complete the development and debugging

of computer software. Bram et al. [29] proposed ChemCrow, which integrates 17 tools designed by experts to enhance the capabilities of LLMs in the field of chemistry. ResearchAgent [3] mimics the steps of humans writing scientific research papers, guiding the LLM to generate complete research ideas including problem identification, method development, and experimental design.

This paper conducts research focusing on typical LLM-driven AIoT applications, namely personal agents. Regarding personal agents, VIVO released Blue Heart V based on the BlueLM [46]; Xiaomi’s Xiao Ai has also integrated LLMs [55]; Google’s Pixel 8 Pro [15] and Bard assistant [17] have both integrated LLMs; and Microsoft has integrated LLM capabilities into Microsoft Copilot [34] and launched Copilot+PC [32]. Furthermore, AppAgent [58] and AutoDroid [50] can execute various tasks on mobile phones by autonomously learning and mimicking human clicking and sliding gestures. This paper mainly focuses on personal agents, aiming to reduce agent response latency by analyzing the similarity between task request texts submitted by users and reusing plans generated by the LLM.

2.2 Caching and Reuse Methods

In computer architecture, caching aims to bridge the gap between memory access latency and processor processing speed [5, 43]. Today, the concept of caching has expanded to various fields and plays an important role in improving computer system performance. Structures located between different types of devices to eliminate the impact of access time differences can be referred to as caches, such as disk caches [27, 31, 52], CDN caches [6, 56, 57], Web caches [36, 47], and container caches [12, 23, 42], etc. The effectiveness of caching stems from the temporal locality of requests, meaning that in computer applications, content that has just been requested is likely to be requested again [45]. Caching recently requested content for reuse in the next request can effectively reduce response latency.

In the past two years, researchers have explored the application of caching and reuse technologies in LLMs. Researchers have conducted research on KV Cache in LLMs. Based on the calculation characteristics of Self-Attention, Kwon et al. [21] proposed PageAttention, a KV Cache management method. Zhang et al. [59] introduced H2O to compress KV Cache based on the varying importance of tokens, while Liu et al. [28] proposed CacheGen to perform dynamic recomputation of KV Cache, addressing system-level challenges when LLMs process long contexts. HotPrefix [25] and IMPRESS [7] further advance KV cache optimization, particularly for scenarios involving shared long contexts or system prompts. Furthermore, several studies [4, 13, 61] have focused on caching and reusing LLM-generated responses based on semantic similarity between request texts. Specifically, Gill et al. [13] proposed a caching and reuse method based on federated learning that considers user privacy and similarity retrieval costs. Zhu et al. [61] provided a theoretical analysis of caching mechanisms for LLM responses, and Fu and Feng [4] developed open-source tools enabling developers to integrate caching and reuse mechanisms into LLMs. In the domain of text-to-image diffusion models, the NIRVANA method [1] reduces iterative denoising steps by reusing intermediate noise states created during previous image generation

This paper mainly focuses on caching and reusing the *plans* generated by LLM-driven agents. Existing methods mainly propose caching and reuse methods for the *responses* of LLMs, which cannot be directly applied to LLM-driven agents.

3 Workflow and Research Objectives of LLM-Driven Agents

This section formally describes the workflow of the LLM-driven agent studied in this paper and briefly introduces the research objectives.

As shown in Figure 1, when the LLM-driven agent studied in this paper serves a user:

- (1) The agent receives a task request q_i submitted by the user, which is a natural language text.
- (2) Upon receiving the request, the agent uses the planning capability of the LLM to generate a plan p_i for q_i .
- (3) After obtaining the plan p_i , the agent executes the plan using various tools such as search engines and code interpreters to obtain the response r_i . The response r_i can be the natural language text desired by the user or a response code returned after task execution (e.g., a 200 code in HTTP services).
- (4) The agent returns the response to the user.

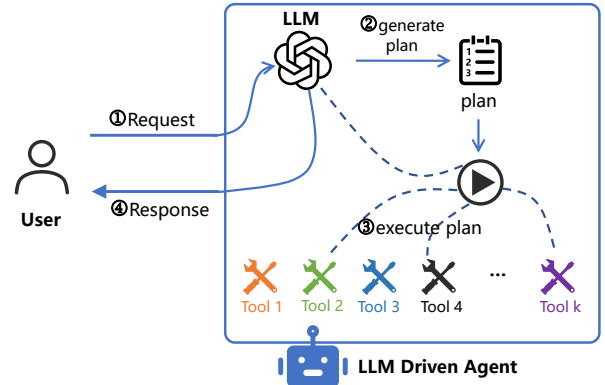


Figure 1: Workflow of LLM-driven agent

Without loss of generality, we define $Q = \{q_1, q_2, \dots\}$, $P = \{p_1, p_2, \dots\}$, and $R = \{r_1, r_2, \dots\}$. For request q_i , the time taken to generate the plan text is referred to as the *plan generation latency*, denoted as t_p . Specifically, t_p refers to the time interval from the moment the agent receives q_i to the moment the plan text p_i generation is completed (i.e., generating the $\langle \text{EOS} \rangle$ token). The time taken to execute the plan is referred to as the *execution latency*, denoted as t_e . This refers to the time interval from the moment the plan text p_i generation is completed to the moment the response r_i is produced. The total response latency for request q_i is $t = t_p + t_e$.

As mentioned earlier, among the task requests received by the agent, approximately 30% are semantically identical or similar. If we can identify identical or similar requests and reuse the plans generated by the agent for previous requests, we can eliminate the plan generation latency t_p , thereby reducing the response latency

and optimizing the user experience. In this paper, requests that can factually reuse previous plans are defined as *reusable requests*, and requests that factually cannot reuse previous plans are defined as *non-reusable requests*.

When defining the similarity between requests, if a non-reusable request is judged as reusable, it will lead to the reuse of an incorrect plan, consequently returning a wrong response to the user. Conversely, if a reusable request is judged as non-reusable, it incurs extra plan generation latency. Therefore, designing a method that can accurately define the similarity between requests is essential.

Furthermore, for reusable requests, when reusing a previously generated plan, certain parameters in the original plan need to be replaced with the relevant parameters from the current request. Currently, there is no method that effectively achieves parameter replacement; that is, plan reuse cannot be effectively completed. Thus, we also need to design a method capable of effectively reusing plans.

This section mainly introduced the workflow of LLM-driven agents, including four parts: receiving user requests, generating plans, executing plans, and returning responses, and provided a formal description. Additionally, this section defined reusable and non-reusable requests, clarifying the importance of designing methods for accurate similarity definition and effective plan reuse.

4 Reuse Mechanism Design

This section introduces the specific content of the reuse mechanism for LLM-driven agents proposed in this paper, **AgentReuse**, and explains the workflow of AgentReuse using examples.

When an LLM-driven agent serves a user, it receives a request q_i , utilizes the LLM’s planning capability to obtain a plan text p_i , and finally uses tools to execute the plan to get a response r_i . The plan is generated by the LLM, and the plan generation latency t_p is proportional to the length of the plan text; the longer the plan text, the larger t_p . Using AutoGen [53] as the agent framework and OpenAI’s GPT-4 API as the LLM, 100 tests were conducted on a real-world dataset [40]. The results indicate that the average plan generation latency is 31.8s. Previous research [13] and our tests on the real-world dataset both indicate that about 30% of requests are identical or similar. For identical or similar requests, if the plan or response of a previous request can be reused, the response latency can be reduced to a certain extent. Based on this, our goal is to design a plan reuse mechanism for LLM-driven agents to reduce response latency.

As shown in Algorithm 1 and Figure 2, AgentReuse utilizes the semantic similarity and differences between request texts to effectively reuse the plans of LLM-driven agents.

Next, we explain AgentReuse using the example q_i = "Book a ticket from Hefei to Beijing for the day after tomorrow." First, we vectorize the request text q_i and perform intent classification. Intent classification refers to classifying request texts into different categories based on their semantics, utilizing semantic similarities between different texts to group them into the same intent category. In personal assistants, intent categories mainly include coarse classifications such as query, play, send, download, and book. The intent of the current example q_i is "Book." Furthermore, slot filling tasks are usually performed jointly with intent classification tasks

Algorithm 1: AgentReuse

Require: Requests $Q = \{q_1, q_2, \dots\}$, Cache $\{c_0, c_1, \dots, c_n\}$, Intent Categories, Similarity threshold γ .

Ensure: Response $R = \{r_1, r_2, \dots\}$.

- 1: Initialize Cache as empty.
- 2: **for** each q_i **do**
- 3: Vectorize q_i to get $E(q_i)$.
- 4: Perform intent classification and parameter identification on q_i , obtaining intent c_i and parameters $para_i$.
- 5: **if** $c_i \neq \text{Undefined Intent}$ **then**
- 6: Extract parameters $para_i$ from q_i to get q_i^- and $E(q_i^-)$.
- 7: Perform similarity search for $E(q_i^-)$ in Cache (within category c_i).
- 8: **if** Highest similarity score $\geq \gamma$ **then**
- 9: Reuse plan p_m from q_m (where $E(q_m^-)$ has the highest similarity with $E(q_i^-)$ and $\geq \gamma$).
- 10: Execute plan p_m using tools with parameters $para_i$ to get response r_i .
- 11: **else**
- 12: Call LLM to generate plan p_i .
- 13: Execute plan using tools to get response r_i .
- 14: Store $\langle E(q_i^-), p_i, c_i \rangle$ into Cache.
- 15: **end if**
- 16: **else**
- 17: Call LLM to generate plan p_i .
- 18: Execute plan using tools to get response r_i .
- 19: **end if**
- 20: **end for**
- 21: **return** Response R .

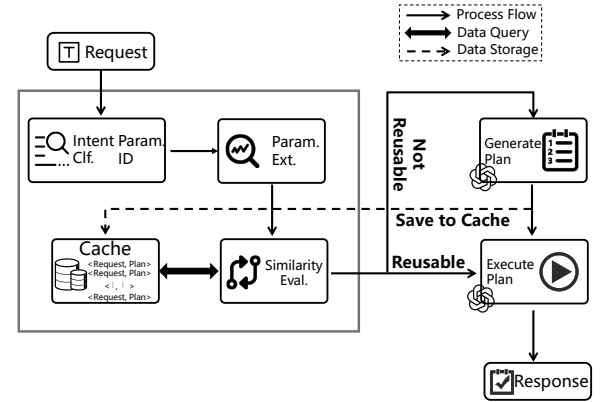


Figure 2: Flowchart of AgentReuse

[30]. Slot filling is used to identify key parameters and temporarily store the parameters "day after tomorrow, Hefei, Beijing" into $para_i$. Subsequently, within the intent category "Book," parameter extraction and similarity definition are performed. For parameter extraction, in the example, the parameters "day after tomorrow, Hefei, Beijing" need to be extracted. Removing these parameters from the request text q_i yields q_i^- = "Book a ticket from to". This allows it to be distinguished from tasks like booking restaurants or meeting rooms within the "Book" class, finding requests more

similar to "Book a ticket from to" and reusing the plans of similar requests. Similarity is defined by the similarity threshold γ . γ is an empirical value, and this paper sets the default value of γ to 0.75.

To effectively reuse plans, when reusing a plan, the previously identified parameters $para_i$ are used as input, utilizing the semantic differences between request texts. When there is no request similar to q_i^- in the cache, the LLM is called to generate a plan, which is then executed using tools to get response r_i , and $\langle E(q_i^-), p_i, c_i \rangle$ is stored in the cache. Here, $E(q_i^-)$ is the vectorized q_i^- , p_i is the plan generated by the LLM, and c_i is the intent category of text q_i . When q_i is not within the "Book" intent category, i.e., the intent category is undefined, the LLM is directly called to generate a plan p_i , which is executed to get the response r_i . It is worth noting that in Figure 2, parameter extraction, similarity definition, and cache lookup are performed within the current intent category, which can reduce search costs.

In AgentReuse, this paper classifies the intent of user requests by utilizing the semantic similarity between request texts, while simultaneously utilizing the semantic differences between request texts to identify and extract key parameters. We use the identified key parameters to perform similarity evaluation and parameter replacement on request texts, achieving accurate similarity definition for requests received by LLM-driven agents and effective reuse of plans. Additionally, since requests submitted by users may involve real-time changing information, such as flight information, this paper reuses the *plan* rather than the *response*.

Compared to the execution process of existing LLM-driven agents, AgentReuse incurs additional costs in terms of storage and latency.

- (1) **Storage Cost:** AgentReuse needs to use models to vectorize texts and classify intents. When the model is running, it occupies VRAM. Vectorized texts and plans generated by the LLM need to be cached, occupying storage space.
- (2) **Latency Cost:** Compared to the existing execution process, the new workflow introduced by AgentReuse brings extra runtime latency. The significant parts of the latency are using models for intent classification and parameter identification, as well as defining the similarity between user requests and cached requests.

It is worth noting that existing personal assistants already need to use models to classify user request intents [20, 38]. Therefore, the extra storage and latency costs brought by intent classification and parameter identification may not be counted towards the cost of AgentReuse. A specific analysis and discussion of the extra costs will be conducted in Section 6.

This section introduced the core content of this paper, the plan reuse mechanism AgentReuse for LLM-driven agents. To define the similarity of requests, AgentReuse performs intent classification and key parameter identification on requests, extracts key parameters, and then performs similarity definition to improve performance. If a reusable plan exists in the cache, it is reused and executed; if not, a new plan is generated and stored in the cache. Finally, this section briefly analyzed the additional costs incurred by AgentReuse. To effectively reuse plans, this paper adopts a method of adding prompt words to obtain structured plans, which will be introduced in the next section.

5 Implementation of the Reuse Mechanism

Based on the plan reuse mechanism for LLM-driven agents, **AgentReuse**, proposed in the previous section, this section aims to introduce the relevant technologies for implementing AgentReuse. It is worth noting that the relevant technologies adopted in this section represent only one method for implementing AgentReuse. For different performance requirements, extra cost limitations, and input text language types, different technical routes can be selected.

5.1 Models and Techniques

For Intent Clf. and Param. ID, this paper adopts the bert-base-chinese model [14]. This model is the Chinese version of the BERT model and possesses powerful language understanding capabilities. The model uses a bidirectional Transformer architecture and is pre-trained on a large amount of Chinese corpus data. It performs excellently on multiple Chinese natural language processing tasks, such as text classification, named entity recognition, and sentiment analysis, providing a solid foundation for Chinese NLP tasks.

For **Text Vectorization**, this paper adopts the m3e-small model [37]. M3E refers to Moka Massive Mixed Embedding. Specifically, this model is trained by MokaAI using a massive dataset of Chinese sentence pairs and supports mixed Chinese-English homogeneous text similarity calculation. It is a text embedding model capable of converting natural language into dense vectors. The m3e-small model has 24 million parameters and generates vectors with a dimension of 512.

For **Text Semantic Similarity Calculation**, this paper employs Cosine Similarity. Vectors can capture the semantic meaning of words or sentences; words or sentences with similar meanings will have similar vector representations. For example, "train" and "high-speed rail," although textually different, are semantically similar. Therefore, after vectorization, the vectors corresponding to these two terms are close in the vector space. Cosine similarity is typically used to quantify the similarity between vectors, calculated by taking the cosine of the angle between two vectors. The range of cosine similarity is -1 to 1, where -1 indicates the two vectors are completely opposite in direction, 0 indicates they are orthogonal, and 1 indicates they are in the same direction. Generally, in text processing, the closer the cosine similarity value is to 1, the more similar the two texts are. For vectors V_1 and V_2 , the cosine similarity is:

$$\cos \theta = \frac{V_1 \cdot V_2}{\|V_1\| \cdot \|V_2\|} \quad (1)$$

For **Vector Similarity Search**, in the technical implementation of this paper, we use the IndexFlatIP index from Facebook AI Similarity Search (FAISS) [33] for efficient similarity search. FAISS is an open-source similarity search library by Facebook that provides efficient similarity search and clustering for vectors, supporting searches at the billion-scale level, and is currently the most mature approximate nearest neighbor search library. It is worth noting that FAISS does not directly provide cosine similarity calculation; the IndexFlatIP selected in this paper performs dot product calculation. Therefore, when performing cosine similarity calculation, the vectors need to be normalized first.

5.2 Structured Plan Reuse

Furthermore, since plans generated by LLM-driven agents like AutoGen are usually natural language texts, to store and effectively reuse plans, this paper performs structured processing on the natural language text plans. In the plan generation prompt words used by AutoGen with the LLM, we prompt the LLM to simply annotate the dependency information between sub-tasks when generating the plan. Based on the dependency annotations, the plan text can be processed to obtain a structured plan. Additionally, in agent frameworks like AutoGen, for system security considerations, tool usage is usually conducted within containers [8, 22, 41]. Therefore, container image information needs to be recorded in the steps for subsequent reuse.

For the natural language text plan p_i of request q_i , the plan is represented as:

- Step 1, Step Description, Container Image, Input, Dependency, Output;
- Step 2, Step Description, Container Image, Input, Dependency, Output;
- ...
- Step k , Step Description, Container Image, Input, Dependency, Output.

Taking the plan generated by "Book a ticket from Hefei to Beijing for the day after tomorrow" as an example, this paper converts it into the following structure (using the abbreviations requested):

- Step 1 Date Query**, query_date, Date Desc., No Dependency, Output: Date.
- Step 2 Weather Query**, query_weather, (Location, Date), Dep: Step 1, Output: Weather.
- Step 3 Flight Query**, query_flight, (Origin, Dest, Date), Dep: Step 1, Output: Flight Info.
- Step 4 QTrain Query**, query_train, (Origin, Dest, Date), Dep: Step 1, Output: Train Info.
- Step 5 Price Comp.**, compare_price, (Flight Info, Train Info), Dep: Step 3, 4, Output: Comparison.
- Step 6 Schedule Query**, query_schedule, Date, Dep: Step 1, Output: Personal Schedule.
- Step 7 Pref. Query** (Recommend Travel Plan), call_prefer, All Info, Dep: Step 2, 5, 6, Output: Recommendation.

Figure 3 illustrates the execution graph of the above plan. For each step, the current step can only be executed after its dependent steps are completed. Specifically, Steps 2, 3, 4, and 6 depend on Step 1; Step 5 depends on Steps 3 and 4; and Step 7 depends on Steps 2, 5, and 6. In actual execution, the steps are implemented as follows: use open APIs to complete Steps 1 and 2; use open APIs or applications like Ctrip and 12306 to complete Steps 3 and 4; use Python code to complete Step 5; use a calendar application to complete Step 6; and finally, invoke the LLM to complete Step 7.

This section introduced the technical methods adopted to implement AgentReuse, including using bert-base-chinese as the intent recognition model, m3e-small as the text vectorization model, and adopting cosine similarity to evaluate text semantic similarity. As mentioned at the beginning of this section, the methods introduced here are just one of the schemes to implement AgentReuse. For different languages, different performance requirements, and

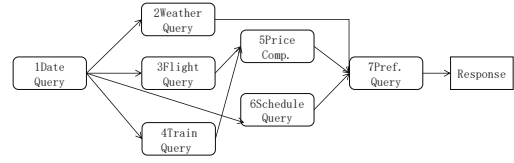


Figure 3: Execution graph for the plan generated by the request

different resource usage limits, different models and related technologies can be selected.

6 Experiments and Result Analysis

To evaluate the effectiveness and extra costs of the proposed mechanism AgentReuse, we conducted a series of experiments based on a real-world dataset. This section first details the experimental setup, then introduces the evaluation metrics, and finally presents the experimental results and analyzes the extra costs. Experimental results show that compared to existing methods, AgentReuse achieves effective plan reuse with an effective reuse rate of 93%. Its F1 score for similarity definition is 6.8 percentage points higher than existing methods, accuracy is 13.06 percentage points higher, and latency is reduced by 60.61 percentage points. Compared to the execution process of existing LLM-driven personal agents, AgentReuse incurs an extra VRAM usage of about 100 MB, memory usage for processing each request does not exceed 1 MB, and latency does not exceed 10 ms.

6.1 Experimental Setup

The experiments in this paper use the SMP dataset [39], which contains 2,664 task requests for LLM-driven personal agents. Examples include text requests like "What are the flight times to Chengdu tomorrow morning" and "Play a poem by Li Qingzhao," covering 23 intent categories such as "LAUNCH," "QUERY," and "ROUTE."

The experiments were conducted on a server with the following configuration: Operating System Ubuntu 20.04.6 LTS, Intel Xeon Gold 6330 CPU @ 2.00 GHz CPU, GPU NVIDIA A100 80 GB, Python version 3.10, CUDA version 11.4, and server memory of 503 GB.

To test the effectiveness of AgentReuse, this paper selected two methods proposed by other researchers as comparison methods (Baselines):

- (1) **GPTCache** [4]: A mechanism for caching and reusing LLM responses. When defining similarity, it directly vectorizes the original request text to calculate semantic similarity.
- (2) **MeanCache** [13]: A mechanism for caching and reusing LLM responses. It performs Principal Component Analysis (PCA) on the request text to reduce dimensions to 64 before performing similarity definition.

In addition, to verify the effectiveness of different modules in AgentReuse (i.e., ablation studies), two variants were tested:

- (1) **OneIntent**: When defining similarity in the implemented AgentReuse, retrieval is performed on all cached content instead of only within the current intent category.
- (2) **WithArgs**: When defining similarity in the implemented AgentReuse, parameter extraction is not performed; instead,

the vectorized original request text is directly used for similarity search.

6.2 Evaluation Metrics

We need to evaluate the performance of AgentReuse’s similarity definition, the effectiveness of plan reuse, extra costs, and performance gains. This section introduces the metrics used to evaluate the above experimental effects respectively.

To evaluate the performance of semantic similarity identification in our caching system, we employ four standard metrics: Precision, Recall, F_β Score (specifically the F1 Score), and Accuracy. These metrics are calculated based on the classification of request reusability into True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). While Precision and Recall measure the system’s ability to correctly identify reusable requests and minimize false identifications, the F1 Score and Accuracy provide a comprehensive assessment of the model’s overall predictive reliability.

This paper uses the effective reuse rate as a quantitative metric to evaluate the effectiveness of plan reuse, detailed in Section 6.3.

For the extra costs of using AgentReuse, this paper uses the extra latency per request and the extra memory occupied (including VRAM) as evaluation metrics.

For the assessment of performance gains, since the use of different reuse mechanisms has no impact on the latency of *executing* the plan, the analysis of performance gains in this section only considers the plan *generation* latency and the extra costs of using the mechanism. It is assumed that all non-True Positive cases require a plan generation.

6.3 Result Analysis

This section mainly introduces the experimental results of similarity definition and plan reuse effectiveness, and analyzes the results.

First, we analyze the experimental results of similarity definition. Only when the similarity definition result is accurate can plan reuse be effective; otherwise, it may lead to consequences such as reusable requests not being judged as reusable, or non-reusable requests being judged as reusable (reusing previous plans), leading to incorrect responses.

Table 1 shows the similarity definition experimental results under the default setting, i.e., similarity threshold $\gamma = 0.75$. The proposed mechanism AgentReuse achieves the highest F1 Score and Accuracy, which are 0.9718 and 0.9459, respectively. Among the comparison methods, the OneIntent variant (which searches all cached content during similarity retrieval instead of just within the current intent category) has performance very close to AgentReuse, but its runtime latency is 17.66% slower than AgentReuse (extra latency costs are discussed later). Compared to GPTCache, which targets LLM response reuse, AgentReuse improves the F1 Score and Accuracy by 6.8 and 13.06 percentage points, respectively.

Next, we adjust the similarity threshold γ , setting it to 0.75, 0.80, 0.85, 0.90, and 0.95, respectively, to observe the performance changes of different methods. Figure 4 (a)-(d) shows the changes in F1 Score, Precision, Recall, and Accuracy for the five methods when the threshold changes. For F1 Score and Accuracy, AgentReuse and its variant OneIntent score higher. Overall, as the threshold

Table 1: Experimental Results for Similarity Evaluation ($\gamma = 0.75$)

Method	F1 Score	Precision	Recall	Accuracy
GPTCache	0.9100	0.9887	0.8428	0.8366
MeanCache	0.6211	0.9991	0.4506	0.4614
OneIntent	0.9712	0.9857	0.9571	0.9444
WithArgs	0.9035	0.9967	0.8262	0.8272
AgentReuse	0.9718	0.9931	0.9513	0.9459

increases, Recall and Accuracy gradually decrease, while Precision gradually increases. This is because when the threshold increases, the reuse mechanism becomes more conservative; only requests with extremely high similarity are judged as reusable, causing originally reusable requests to be judged as non-reusable, leading to a significant drop in Recall.

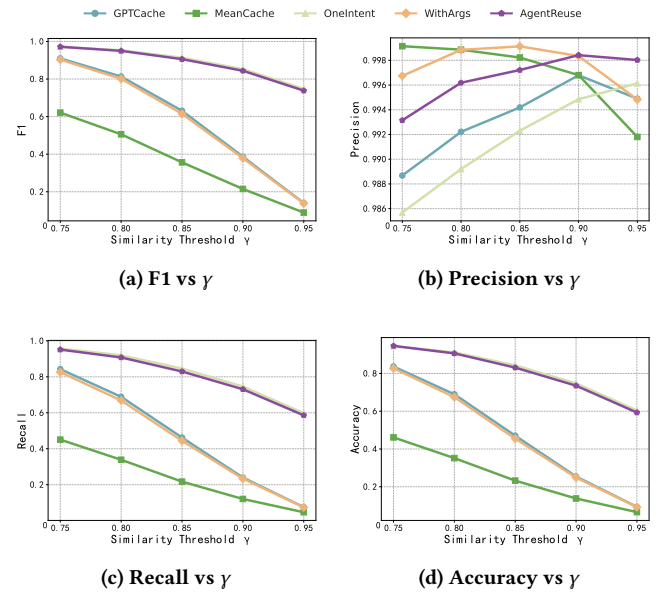


Figure 4: Effect of variation in similarity threshold (γ) on the performance of different methods. (a) F1 score vs γ , (b) Precision vs γ , (c) Recall vs γ , (d) Accuracy vs γ .

Additionally, from Table 1 and Figure 4, we find that the WithArgs variant of AgentReuse performs poorly. WithArgs does not extract key parameters from the original request during similarity definition. Figure 5 shows the detailed similarity judgment comparison between AgentReuse and WithArgs for the same requests when the similarity threshold $\gamma = 0.75$. "Should Reuse" indicates that the request can factually reuse the plan generated by a previous request; "Should Not Reuse" indicates otherwise. For clarity, only results for Request IDs 100 to 500 are selected. As seen in Figure 5, for the same request, because AgentReuse performs parameter extraction (removing dissimilar parts, i.e., key parameters, from similar sentences), AgentReuse obtains higher similarity scores, thereby achieving higher Recall (0.9513 vs 0.8262).

To analyze whether key parameter extraction is effective for structured plan reuse, we conducted experiments using AutoGen

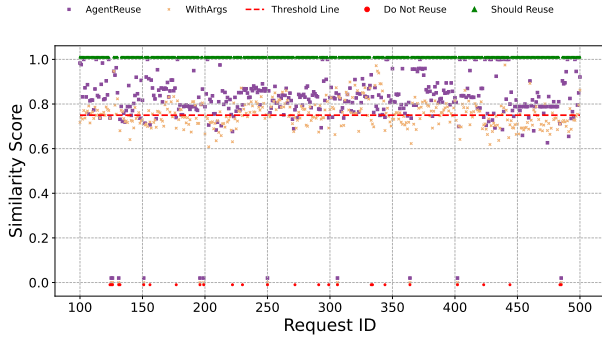


Figure 5: Comparison of similarity scores calculated by AgentReuse and WithArgs

as the agent framework. The experimental standard was to execute two tests for the same user request: the first test used AgentReuse (extracting parameters, reusing the stored structured plan, executing it to get a response); the second test submitted the request directly to AutoGen to generate and execute a plan. Comparing the responses of the two tests, if the results were the same, parameter extraction and plan reuse were considered effective. We conducted experiments on 20 requests, repeating the experiment 5 times for each request (total 100 tests). In 93 tests, the responses were identical, indicating an effective reuse rate of 93%. The 20 requests included 4 booking requests, 4 app launch requests, 2 query requests, 2 translation requests, 2 creation requests, 2 search requests, 2 download requests, 1 navigation request, and 1 undefined request.

This section analyzed the experimental results. The results show that when using the AgentReuse mechanism for similarity definition ($\gamma = 0.75$), the F1 Score reaches 0.9718 and Accuracy reaches 0.9459, both higher than comparison methods. Additionally, we analyzed the changes in metrics with varying thresholds and the effectiveness of removing key parameters. Finally, we analyzed the effectiveness of reuse.

6.4 Extra Cost Analysis

Compared to the execution process of existing LLM-driven agents, AgentReuse uses additional models, occupies extra storage space for caching, and correspondingly introduces extra latency. This section aims to demonstrate through experimental testing that the extra costs introduced by AgentReuse are acceptable.

6.4.1 Storage Extra Cost Analysis. 1) **Model VRAM Usage:** The bert-base-chinese model used for intent classification and parameter identification has about 102.3 million parameters. Each parameter occupies 4B, theoretically occupying about 390.24 MB of VRAM. Experimental tests show the system allocated 444.00 MB, and actual usage was 391.16 MB. The m3e-small model used for vectorization has 24 million parameters, theoretically occupying about 91.55 MB. Tests show system allocation of 102.00 MB and actual usage of 92.12 MB. Although there is extra overhead during execution, the total VRAM usage remains at the MB level, which is acceptable for modern smartphones and other devices.

2) **Vectorized Text Usage:** Typically, vectorized text is stored as 32-bit floating-point numbers (4B each). m3e-small generates

512 dimensions, so each vectorized text occupies 2 KB. Including intent categories and FAISS index overhead (tested to be < 1 MB), the extra memory cost for each request generally does not exceed 1 MB.

3) **Plan Text Usage:** AgentReuse stores plans in a structured format. Using UTF-8 encoding (Chinese characters occupy 3-4B, English/numbers 1B), the formatted plan described in Section 4 has 303 characters and occupies 705 bytes. On average, the extra storage overhead per plan text does not exceed 1 KB.

6.4.2 Latency Extra Cost Analysis. For the reuse mechanisms of LLM-driven agents, the extra latency cost for MeanCache, OneIntent, WithArgs, and AgentReuse consists of intent classification, similarity search, and other latencies. GPTCache does not classify intent, so its latency consists of similarity search and other latencies. Other latencies mainly include text vectorization and storing content in the cache. Figure 6 shows the composition of latency costs per request for the 5 methods after 50 tests.

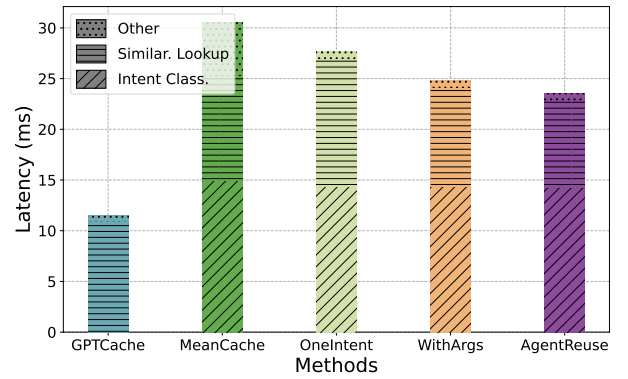


Figure 6: Composition of latency of different methods to process each request

Figure 6 shows that GPTCache has the lowest total latency per request (11.528 ms) because it does not use an intent classification model, but its similarity definition performance is poor as mentioned in Section 6.3. Among methods using intent classification, AgentReuse has the lowest total latency (23.489 ms). It can be seen that AgentReuse and OneIntent have close performance in F1 Score and Accuracy, but AgentReuse has lower latency cost. OneIntent (a variant of AgentReuse) has 17.66% higher latency than AgentReuse. The intent classification latency is similar, but OneIntent’s similarity search latency is 47.94% higher than AgentReuse because it searches all cached content, increasing the search space. Additionally, searching across all content leads to identifying some non-reusable requests as reusable, decreasing precision.

Furthermore, we observe that for the 4 methods excluding MeanCache, “other latencies” do not exceed 1 ms (all are 5.228 ms). MeanCache uses a PCA model, incurring extra latency for each request. For AgentReuse, the latency for formatting the plan and injecting parameters into the reusable plan was measured to be less than 1 ms on average, which is ignored in subsequent analysis.

In summary, the main extra costs of AgentReuse are approximately 500 MB of VRAM, less than 1 MB of memory for storing each vectorized text and structured plan, and 23.489 ms of latency per

request. Considering that existing personal assistants already have intent classification models, AgentReuse would bring about 100 MB of extra VRAM cost and about 9.206 ms of processing latency per request. Reusing plans generated by agents not only saves plan generation latency but also reduces VRAM usage for running the LLM (typically >5 GB). Moreover, if using paid APIs like OpenAI, it saves costs. Therefore, the extra costs brought by AgentReuse are acceptable.

6.5 Performance Gain Analysis

This section analyzes the performance gains of using AgentReuse compared to using no reuse mechanism and using GPTCache. Since the reuse mechanism does not affect plan *execution* latency, we only consider plan generation latency and the extra costs of the mechanism. We assume a plan generation latency of 31.8s and that all non-True Positive cases require plan generation.

In the experiment with AgentReuse ($\gamma = 0.75$), there were 2,464 requests that were True Positives (i.e., actually reusable and identified as such). For the 2,644 requests in the dataset, if all required LLM calls (no reuse), the latency would be 84,079.2s. If using AgentReuse (23.489 ms per request), processing 2,644 requests takes 62.1s. Generating plans for the 180 non-True Positive requests takes $180 \times 31.8s = 5,724s$. Thus, the total latency for AgentReuse is 5,786.1s. For GPTCache, there were 2,183 True Positives and 461 non-True Positives, resulting in a total latency of 14,690.28s. Therefore, compared to no reuse, AgentReuse reduces latency by 93.12%. Compared to the LLM response caching method GPTCache, AgentReuse reduces latency by 60.61%.

This section used the SMP dataset for experiments and analyzed the results. The results show that AgentReuse and its variant OneIntent outperform existing methods in similarity definition. Analysis of extra costs indicates that AgentReuse adds about 100 MB of VRAM, <1 MB of memory per request, and <10 ms latency compared to existing agent processes. Performance gain analysis shows that considering extra costs, AgentReuse reduces latency by 93.12% compared to no reuse and 60.61% compared to GPTCache.

7 Discussion and Future Work

7.1 Multi-Intent Classification

The main research object of this paper is the LLM-driven agent, specifically LLM-driven AIoT applications, such as Xiaomi's Xiao Ai and OPPO's Xiaobu Assistant. Traditionally, a typical user produces only one input to the agent, such as "Book a ticket from Hefei to Beijing for the day after tomorrow." Nowadays, with the development of society, smart cars are gradually entering households. Smart cars have more components, and at this time, a user's single sentence may contain multiple tasks, such as "Lower the front right window, raise the rear left window, turn on the wipers, and play music by David Tao." In this case, a multi-intent classification model needs to be adopted to classify intents and perform Param. ID in the text. After classification, the sentence is split into multiple parts, and then the AgentReuse mechanism is used for reuse in parallel. However, when there is a causal relationship between sentences, such as "If the indoor temperature exceeds 28 degrees Celsius, turn the air conditioner to cooling at 26 degrees Celsius," the sentence cannot be simply split into multiple parts. A new model needs to

be introduced to process sentences with causal relationships, and then AgentReuse can be used to reuse previously generated plans.

7.2 Agent Runtime Environment under Serverless Computing Architecture

Serverless computing, also known as server-imperceptible computing, is capable of achieving fine-grained resource management and rapid service scaling by executing tasks using containers [9, 19], and is regarded as one of the efficient computing paradigms for executing LLMs [2, 11]. Furthermore, as mentioned in Section 5, in LLM-driven agents, tool usage is also typically conducted within containers for system security considerations. Therefore, constructing an agent runtime environment under a serverless computing architecture is one of the future works of this paper. Regarding the reuse mechanism designed in this paper, converting unstructured texts into structured execution steps presents certain challenges when implementing effective plan reuse. When running agents under a serverless computing architecture, one can utilize system-level container execution tracking commands to trace input and output dependencies and execution orders among multiple containers executing tools. This allows for constructing more accurate formatted plan execution steps, thereby improving the effective plan reuse rate.

8 Conclusion

In today's era of artificial intelligence, agents are considered a promising path towards Artificial General Intelligence (AGI). This paper proposes and implements AgentReuse, a mechanism that can effectively reuse plans generated by LLM-driven agents, and tests the effectiveness and extra costs of this mechanism on a real-world dataset. Experimental results show that compared to existing methods, AgentReuse performs better in defining similarity between requests. By storing plan texts in a structured format, it achieves effective reuse of plans. Compared with not using a reuse mechanism, it can reduce latency by 93.12%. In actual real-world operating environments, the tasks faced by agents are often more complex. As an exploratory work, the AgentReuse designed and implemented by us may not be able to handle all situations and needs to be modified and improved by combining characteristics from more practical scenarios. Today, LLMs have integrated into our lives, but LLM-driven agents have not yet been applied on a large scale. We hope this work can help promote the practical landing of agent applications, especially LLM-driven AIoT applications.

References

- [1] Shubham Agarwal, Subrata Mitra, Sarthak Chakraborty, Srikrishna Karanam, Koyel Mukherjee, and Shiv Kumar Saini. Approximate caching for efficiently serving {Text-to-Image} diffusion models. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 1173–1189, 2024.
- [2] Amazon Web Services. AWS and Hugging Face collaborate to make generative AI more accessible and cost efficient. <https://aws.amazon.com/blogs/machine-learning/aws-and-hugging-face-collaborate-to-make-generative-ai-more-accessible-and-cost-efficient/>, 2024. [Accessed: 2024-04-01].
- [3] Jinheon Baek, Sujay Kumar Jauhar, Silviu Cucerzan, and Sung Ju Hwang. Researchagent: Iterative research idea generation over scientific literature with large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6709–6738, 2025.
- [4] Fu Bang. Gptcache: An open-source semantic cache for llm applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 212–218, 2023.
- [5] Anubhav Bhatla, Biswabandan Panda, et al. The maya cache: A storage-efficient and secure fully-associative last-level cache. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 32–44. IEEE, 2024.
- [6] Jiayi Chen, Nihal Sharma, Tarannum Khan, Shu Liu, Brian Chang, Aditya Akella, Sanjay Shakkottai, and Ramesh K Sitaraman. Darwin: Flexible learning-based cdn caching. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 981–999, 2023.
- [7] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. {IMPRESS}: An {Importance-Informed} {Multi-Tier} prefix {KV} storage system for large language model inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 187–201, 2025.
- [8] A Chu and C Shoemaker. Tutorial: Use code interpreter sessions in semantic kernel with Azure container apps. <https://learn.microsoft.com/en-us/azure/container-apps/sessions-tutorial-semantic-kernel>, 2024.
- [9] Shuiguang Deng, Hailiang Zhao, Zhengzhe Xiang, Cheng Zhang, Rong Jiang, Ying Li, Jianwei Yin, Schahram Dustdar, and Albert Y Zomaya. Dependent function embedding for distributed serverless edge computing. *IEEE Transactions on Parallel and Distributed Systems*, 33(10):2346–2357, 2021.
- [10] Xin Luna Dong, Seungwhan Moon, Yifan Ethan Xu, Kshitiz Malik, and Zhou Yu. Towards next-generation intelligent assistants leveraging llm techniques. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 5792–5793, 2023.
- [11] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. {ServerlessLLM}:{Low-Latency} serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 135–153, 2024.
- [12] Alexander Fuerst and Prateek Sharma. Faascache: keeping serverless computing alive with greedy-dual caching. In *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*, pages 386–400, 2021.
- [13] W Gill, M Elidrisi, P Kalapatapu, et al. Privacy-aware semantic cache for large language models. *arXiv preprint arXiv:2403.02694*, 2024.
- [14] Google. bert-base-chinese. <https://huggingface.co/google-bert/bert-base-chinese>, 2024.
- [15] Google. Google pixel 8 pro. https://store.google.com/product/pixel_8_pro, 2024.
- [16] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations*, 2023.
- [17] S Hsiao. Assistant with Bard: A step toward a more personal assistant. <https://blog.google/products/assistant/google-assistant-bard-generative-ai/>, 2024.
- [18] Huawei Device Co., Ltd. Wake up to intelligent voice. <https://consumer.huawei.com/cn/emui-11/tips/smart-home-list/article5/>, 2024.
- [19] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. Cloud programming simplified: A berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383*, 2019.
- [20] Y Kinza. Virtual assistant (ai assistant). <https://www.techtarget.com/searchcustomerexperience/definition/virtual-assistant-AI-assistant>, 2024.
- [21] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [22] LangChain. Security. <https://python.langchain.com/v0.1/docs/security/>, 2024.
- [23] Guopeng Li, Haisheng Tan, Xuan Zhang, Chi Zhang, Ruiting Zhou, Zhenhua Han, and Guoliang Chen. Online container caching with late-warm for iot data processing. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pages 1547–1560. IEEE, 2024.
- [24] Yuanchun Li, Hao Wen, Weijun Wang, et al. Personal LLM agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- [25] Yuhang Li, Rong Gu, Chengying Huan, Zhibin Wang, Renjie Yao, Chen Tian, and Guihai Chen. Hotprefix: Hotness-aware kv cache scheduling for efficient prefix sharing in llm inference systems. *Proceedings of the ACM on Management of Data*, 3(4):1–27, 2025.
- [26] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of {LLM-based} applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 929–945, 2024.
- [27] Yubo Liu, Yuxin Ren, Mingrui Liu, et al. Optimizing file systems on heterogeneous memory by integrating DRAM cache with virtual memory management. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies*, pages 71–87, 2024.
- [28] Yuhang Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024.
- [29] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024.
- [30] Ziyu Ma, Bin Sun, and Shutao Li. A two-stage selective fusion framework for joint intent detection and slot filling. *IEEE Transactions on Neural Networks and Learning Systems*, 35(3):3874–3885, 2024.
- [31] S McAllister, B Berg, J Tutuncu-Macias, et al. Kangaroo: Caching billions of tiny objects on flash. In *Proceedings of the 28th Symposium on Operating Systems Principles*, pages 243–262, 2023.
- [32] Y Mehdi. Introducing Copilot+ PCs. <https://blogs.microsoft.com/blog/2024/05/20/introducing-copilot-pcs/>, 2024.
- [33] Meta. FAISS. <https://ai.meta.com/tools/faiss/>, 2024.
- [34] Microsoft. Microsoft copilot. <https://copilot.microsoft.com/>, 2024.
- [35] Mindstream. AutoGPT. <https://autogpt.net/>, 2024.
- [36] A S Mirheidari, S Arshad, K Onarlioglu, et al. Cached and confused: Web cache deception in the wild. In *Proceedings of the 29th USENIX Security Symposium*, pages 665–682, 2020.
- [37] MokaAI. M3E models. <https://huggingface.co/moka-ai/m3e-small>, 2024.
- [38] NVIDIA. Intelligent virtual assistant. <https://www.nvidia.com/en-us/ai-data-science/ai-workflows/intelligent-virtual-assistant/>, 2024.
- [39] Chinese Information Processing Society of China. The evaluation of chinese human-computer dialogue technology (smp2019). <https://conference.cipsc.org.cn/smp2019/evaluation.html>, 2019.
- [40] Open-Assistant. Open assistant conversations dataset release 2. <https://huggingface.co/datasets/OpenAssistant/oasst2>, 2024.
- [41] G Shishir Patil, Tianjun Zhang, Vivian Fang, et al. Goex: Perspectives and designs towards a runtime for autonomous LLM applications. *arXiv preprint arXiv:2404.06921*, 2024.
- [42] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. Icebreaker: Warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 753–767, 2022.
- [43] Ali Sedaghati, Milad Hakimi, Reza Hojabr, and Arrvinth Shriraman. X-cache: a modular architecture for domain-specific caches. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 396–409, 2022.
- [44] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.
- [45] S Traverso, M Ahmed, M Garetto, et al. Temporal locality in today’s content caching: Why it matters and how to model it. *ACM SIGCOMM Computer Communication Review*, 43(5):5–12, 2013.
- [46] VIVO. BlueLM-7B-Chat. <https://huggingface.co/vivo-ai/BlueLM-7B-Chat>, 2024.
- [47] Huan Wang, Kui Wu, Jianping Wang, and Guoming Tang. Rldish: Edge-assisted qoe optimization of http live streaming with reinforcement learning. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, pages 706–715. IEEE, 2020.
- [48] Ji-Liang Wang, Shuai Tong, Xiang-Yang Li, Zheng Yang, Fu Xiao, and Yun-Hao Liu. On the scalability of internet of things systems. *Journal of Computer Science and Technology*, 40(5):1182–1194, 2025.
- [49] Lei Wang, Chen Ma, Xueyang Feng, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):1–26, 2024.
- [50] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yuhao Liu, Yaqin Zhang, and Yunxin Liu. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*, pages 543–557, 2024.
- [51] Lilian Weng. LLM powered autonomous agents. <https://lilianweng.github.io/posts/2023-06-23-agent>, 2023.

- [52] L D Wong, Hao Wu, C Molder, et al. Baleen: ML admission & prefetching for flash caches. In *Proceedings of the 22nd USENIX Conference on File and Storage Technologies*, pages 347–371, 2024.
- [53] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- [54] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwon Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *Science China Information Sciences*, 68(2):121101, 2025.
- [55] Xiaomi Corporation. Xiaoi tongxue. <https://xiaoi.mi.com/>, 2024.
- [56] Gang Yan and Jian Li. Towards latency awareness for content delivery network caching. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, pages 789–804, 2022.
- [57] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, et al. FIFO queues are all you need for cache eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 130–149, 2023.
- [58] Chi Zhang, Zhao Yang, Jiaxuan Liu, et al. AppAgent: Multimodal agents as smartphone users. *arXiv preprint arXiv:2312.13771*, 2023.
- [59] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- [60] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- [61] Banghua Zhu, Ying Sheng, Lianmin Zheng, Clark Barrett, Michael I Jordan, and Jiantao Jiao. On optimal caching and model multiplexing for large model inference. *arXiv preprint arXiv:2306.02003*, 2023.